Cryptography Class Notes

ver. 2015/04/30

These notes are for the first few classes of a course taught from Hoffstein Pipher and Silverman's 'An Introduction to Mathematical Cryptography'. which is herein refereed to as 'the text'. They draw largely from the text but reorder it a bit: bringing Sections 1.7 and 2.5 forward. I number the sections to try to reflect the this.

This is an upper level undergraduate first class in Cryptography.

1 An Introduction to Cryptography

This class is about *Cryptography*, a subfield of *Coding Theory*. For context, Coding Theory is the study of *codes*. The main types of codes are:

i). *Secret codes* - used to transfer information so that only the intended recipient can read it.

ii). *Compression codes* - used to transfer information efficiently.

iii). *Error correcting codes* - used to correctly transfer information that might get corrupted in transfer.

'Cryptography' is usually taken to be the study of secret codes, and this is what we will study. 'Coding theory' can refer to everything, or to the study of the last two types of codes.

1.1 Intro to the Intro: Simple Substitution Cipher

Example 1.1. Say Alice wanted to send a message to Bob, but didn't want Eve to be able to read it.

Tomorrow at Lunch

She could replace every letter with the letter to the left of it on the keyboard, and instead send

Ypzpttpe sy Aimvj

Eve can't read her message, but Bob knows he has to replace every letter with the letter to the left of it on the keyboard, and can convert it back to the original message.

ver. 2015/04/30

What Alice and Bob have used is called a *simple substitution cipher*. It is an example of a scheme called a *cryptosystem*. The elements of a cryptosystem are, informally:

- The message 'tomorrow at lunch' is the *plaintext*.
- The message 'YPZPTTPE SY AIMVJ' is the *ciphertext*.
- The algorithm for converting between ciphertext and the plaintext is called the *key* or *cipher*.
- The process of converting the plaintext to the ciphertext is called *encryption*.
- The process of converting the ciphertext to the plaintext using the key is called *decryption*.
- The process of converting the ciphertext to the plaintext without the key is called *cryptanalysis*, or *cracking* the cipher.

Simple substitution ciphers are and obvious cryptosystem and have historically been very useful. But they can be cracked quite easily. Especially for long messages.

1.1.1 Cracking a Simple Substitution Cipher

The most obvious approach is to try every possible substition of the letters and see which gives some proper English words. But 26! is a pretty big number and this would be tricky even with computers. This is the approach, but one has to be a bit more clever. Just a bit though, this approach is still quite obvious. It appears in Poe's 'The Gold Bug'.

The letter 'e' occurs most frequently in the English language, accounting for about .13 of all letters. Then comes 't' at about .10 and then 'a' at about .08. Given a long enough ciphertext, the basic idea is to count the occurences of each symbol, replace the most frequent with an 'e', the next most frequent with a 't', etc.

Sure this isn't going to work out exactly all the time. Indeed it usually won't. But the most common letters are 'e, t, a, o, n'. Try putting these randomly into the 8 or so most frequent ciphertext symbols, you have a much more managable number of combinations. You can check these for agreement with words in the english language: there should be no two letter words 'ta' or 'ea'. Much better.

One can also use the frequency of two letter combinations. The most common two letter combinations, or digraphs, in English are 'th' 'he' and 'in'. Choosing 't' and 'e' in the first step, if there is some ciphertext letter X such that tX and Xe are frequent, you can be pretty sure that this is an h.

ver. 2015/04/30

And if you have a three letter work 'PYQ' occuring several times you can be pretty sure this 'the' or 'and'.

With a bit of math, but more contextual reasoning, it is not hard to solve a simple substition cipher. But it is hard enough to be fun. That is why there are Word Jumbles. http://www.wordjumble.com/ Give them to your Dad and tell him they fight off Alzheimier's.

Problem 1.1. Reading Section 1.1.1 of the text if necessary, do Problem 1.4 (a) from the text

1.1.2 Private vs. Public key Cryptosystems

In the simple substitution cipher, it is required that Alice and Bob both know the key. Such cyptosystems are called *private key* cryptosystems, or *symmetric* cryptosystems.

Contrastingly, in a *public key* cryptosystem, or *asymmetric* cryptosystem, there are two keys, one for encryption, and one for decryption. In such a cryptosystem, Bob keeps the decryption key secret, but makes the encryption key public. Under this scheme, anyone, including Alice, can send Bob a message, which only Bob can read.

There are advantage to both private key and public key systems.

As we saw, private key cryptosystems can be cracked, but this is not really one of their shortcomings. There are many different private key cryptosystems, and it is not hard to come up with one that is impossible to crack without the key.

Problem 1.2. Come up with at private key cryptosystem that is impossible to crypoanalyse, and explain why it is impossible.

But this suggests one of the shortcomings. What if Bob loses the key? Wiki up the story of the Enigma code. Those Germans were using a private key cryptosystem called the Enigma - a fancy sort of shift cipher with a progressive shift. They were sneaking around in their submarines wreaking German havoc. Nobody could crack their code. Not even the guy from Rubicon. But then the Americans or the British, or probably the Canadians, caught one of their submarines and got the key. End of story, Germans.

Actually, it turns out it was the Polish that got the Enigma Machine from the Germans, and that that wasn't the end of the story. The Enigma Machine wasn't the key to the German code, it was the encryption schema. The key changed everyday, and even with the machine, the English had to tried a probability attack to crack the key. This took to long to be useful, until Alan Turing build the first computer, specifically to crack the key. Everyone should watch the beautiful movie, 'The Imitation Game'.

ver. 2015/04/30

If you want a code that many people can use, and any one person loses the private key, the code is compromised.

In a public key cryptosystem, anyone can have the encryption key. Alice doesn't care if Bob number 17 gives it up to Eve, she even makes it public, Eve still can't decode other peoples ciphertexts.

Another shortcoming of the private key cryptosystem is that Alice and Bob must first meet to share the key, and this is not always practical.

While there are uncrackable private key cryptosystems, public key cryptosystems, by necessity, can be cracked. What makes one good, is that it is difficult to crack. In this course, except in the first few classed, will look exclusively at public key cryptosystems, and the mathematics behind them.

1.7 A more rigourous introduction

This is mostly Section 1.7 of the text. We make our definitions more rigourous, and lay out the rules of the game- those assumptions that we will be making to allow us to get to the math of the subject.

1.7.1 Encryption Schemes

A *cipher* or *encryption scheme* is a tuple $(\mathcal{K}, \mathcal{M}, \mathcal{C}, e, d)$ where $\mathcal{M}$ is a space of *plaintexts* or messages, $\mathcal{C}$ is a space of *ciphertexts*, $\mathcal{K}$ is a space of *keys*, e is function

$$e : \mathcal{K} \times \mathcal{M} \to \mathcal{C},$$

called *encryption*, and d is a function

$$d : \mathcal{K} \times \mathcal{C} \to \mathcal{K}$$

called *decryption*, such that $d(k, e(k, m)) = m$ for any key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$.

This is more notation that is useful for our purposes. Usually we assume that the encryption scheme is understood and for a fixed key k write the encryption and decryption functions

$$e_k : \mathcal{M} \to \mathcal{C} : m \mapsto e(k, m)$$

and

$$d_k : \mathcal{C} \to \mathcal{M} : c \mapsto d(k, c).$$

For public key cryptosystems, we said there is an encryption key k and a decryption key k_d. However one can always compute k_d from k, so we really have only one key. The differce between public and private key systems is that for public key systems, k_d is hard to compute from k.

ver. 2015/04/30

Informally, we will often call e_k the encryption key, and d_k the decryption key, thinking of the key as the function, rather than the number it is based on. When we analyse an encryption scheme, we assume that the enemy, Eve, knows the encryption scheme, but not the (decryption) key.

Clearly, to have $d_k(e_k(m)) = m$, e must be injective and d must be surjective. Other properties that we want of a cipher are

P1) $e_k(m)$ is easy to compute for $k \in \mathcal{K}$ and $m \in \mathcal{M}$.

P2) $d_k(c)$ is easy to compute given k and c.

P3) Given many $c_1, \ldots, c_n \in \mathcal{C}$, it is difficult to compute any of the $d_k(c_i)$ without having k.

P4) Given several pairs $(c_1, d_k(c_1)), \ldots, (c_n, d_k(c_n))$, it is difficult to compute $d_k(c)$ for some $c \notin \{c_1, \ldots, c_n\}$.

Having a pair $(c_i, d_k(c_i))$ is equivalent to having $(e_k(m), m)$ for some $m \in \mathcal{M}$, because we know $e_k(m)$ decodes to m.

Properties P3 and P4 essentially say that cryptoanalysis is hard.

1.7.2 Plaintexts and Ciphertexts as number strings

In the substitution cipher example we saw earlier, the key was a little hard to describe: move once key to the right on the keyboard. Here is more how we will do a substitution cipher. It is called a simple shift substitution cipher.

Associating the ith letter of the alphabet with the number $i-1$, (so $a = 0$ and $b = 1$ and, ..., $z = 25$,) we can write our message " Tomorrow at Lunch" as

$$19\,14\,12\,14\,17\,17\,14\,22 \qquad 00\,19 \qquad 11\,20\,13\,02\,07.$$

Now to encrypt it we pick a secret key $k \in \{0, \ldots, 25\}$ and add it to each letter, modulo 26. Taking key $k = 9$ we encode to

$$03\,23\,21\,23\,01\,01\,23\,06 \qquad 09\,03 \qquad 20\,04\,22\,11\,16.$$

Now perhaps we would use the same letter to number key, and send this as

Dxvxbbxgjduewlq,

but let's just let that be overhead, and always consider our plaintext and ciphertext as numbers.

In this example, our encryption scheme is the 'simple shift cipher'. This tells us that the $\mathcal{M}, \mathcal{C}$, and $\mathcal{K}$ are all $\mathbb{Z}_{26}$ the integers modulo 26. Our actual

ver. 2015/04/30

message consisted of a string of 15 plaintexts in $\mathcal{M}$ which we encrypt and decrypt individually, and the spaces between them.

Having $k = 9$ our encryption key is

$$e_k : m \mapsto m + 9 \text{ modulo } 26$$

and our decryption key is

$$d_k : m \mapsto m - 9 \text{ modulo } 26.$$

This is a symmetric cryptosystem, as d_k is easy to deduce from k or e_k.

We will usually encode messages as binary strings. So with that previous example could encode

$$19\ 14\ 12\ 14\ 17\ 17\ 14\ 22 \qquad 00\ 19 \qquad 11\ 20\ 13\ 02\ 07.$$

into length 5 binary strings as:

$$10011\ 01110\ 01100\ 01110\ 10001\ 10001\ 01110\ 10110 \qquad 00000\ \ldots$$

We didn't apply e to the whole message at once, but to each letter at a time. This is weak, in that is does not satisfy property P4. This is because, $\mathcal{M}$ is too small. Once we know the encoding of each letter we can decode the message. Generally we will string several letters together to get an element of $\mathcal{M}$. For example, we would encode the above message as length 20 binary strings:

$$10011011100110001110100 01\ 10001011101011011111100000\ \ldots$$

where we have encoded the empty space as 11111. This is also considered a part of the encoding, and not of the encryption scheme.

The length B of the strings in $\mathcal{M}$ is called the *blocksize* of the cipher. Often we have to pad the final plaintext block so that the message fits into it.

Generally $\mathcal{M}, \mathcal{C}$, and $\mathcal{K}$ are all regular length binary strings. We use B_m, B_c and B_k to refer to their respective blocklengths.

It is generally accepted that with present computing power, a cipher is safe against an exhaustive search attack if the blocklength is at least 80.

More Examples of Symmetric Ciphers

Let p be a large prime ($p \approx 2^{160}$ so the binary representation is about 160 digits), and $\mathcal{M} = \mathcal{C} = \mathcal{K}$ be the integers modulo p. We will study these a bit later, but for now, if you are unfamiliar with them, just pretend that you are not.

ver. 2015/04/30

Let encryption be

$$e_k(m) = k \cdot m.$$

Then

$$d_k(m) = k^{-1} \cdot m$$

is decryption.

Given k, we will see that it takes about $3\log_2(p) = 320$ calculations to find k^{-1}. A computer can do this in seconds, so this is easy. (We don't have to find k^{-1} for each decryption, it is part of the key that we only have to generate once. So it isn't one of the properties P1 - P4 we mentioned. But it is desirable– it is inconvenient if it takes too long to generate a key.)

Encryption and decryption are easy, so we have properties P1 and P2. Since e_k is surjective, whatever k is, some message encodes to the ciphertext c_i, so having a couple ciphertexts doesn't help us decrypt anything. So we have property P3. However we don't have property P4. As soon as you have some pair $(c, m = d_k(c))$, you know that $c = km$ so you can compute

$$c \cdot m^{-1} = km \cdot m^{-1} = l.$$

So while this satisfies P3, and is secure if Eve only sees encrypted messages, as soon as she knows what a message encodes to, then she can crack it.

One can replace e_k with any affine shift:

$$e_{(k_1,k_2)} : m \mapsto k_1 \cdot m + k_2.$$

Such so-called affine ciphers all fail the same *chosen plaintext attack*.

We now take a brief detour into Algoritmic Complexity before returning to this section.

2.5 Algorithm Complexity

This is the complexity theory we are using for the course. It draws some from Section 2.5 of the text, but adds other things.

The most important thing about an asymmetric encryption scheme is that it is easy to encrypt, easy to decrypt with the decryption key, but hard to decrypt without the key. We have to make the words 'easy' and 'hard' more clear. For this, we talk about computational complexity.

2.5.1 Order Notation

Where f and g are positive functions of x, we say that *'f is of order at most g'*, or *'f is big oh of g'* and write $f = O(g)$ if there exist constants c and N such that

$$(x > N) \Rightarrow (f(x) < c \cdot g(x)). \tag{1}$$

ver. 2015/04/30

It is easy to show that if

$$\lim_{x\to\infty} \frac{f(x)}{g(x)} = c,$$

then (1) holds for some N.

So, for example, $x^2 = O(x^2)$ and $x^2 = O(x^3)$, but $x^3 \neq O(x^2)$. Those are sillyish examples. One of the point of writing something like this is to simplify a function, to get rid of unimportant terms. The typical example is one such as

$$1/2x^2 - 5x + 7 = O(x^2).$$

Another use of the notation is to compare functions the eventual growth of functions. Exponentials grow faster than polynomials, so

$$f(x) = 34x^{100} + 5x^2 + 100000 = O(e^x) \text{ and } e^x \neq O(f(x)).$$

We say that exponentions have greater order than polynomials. There is a heirarchy of orders, those orders that we are most interested in are

logorithms < roots < polynomial < exponentials

Problem 2.3. Show that $\sqrt{x} = O(x)$, and that $x^2 = O(2^x)$. (Hint: l'Hopital might be useful).

Problem 2.4. Show that if $f(x) = O(x^2)$ and $g(x) = O(\log(x))$, then $f(x) + g(x) = O(x^2)$, and $f(x) \cdot g(x) \neq O(x^2)$.

2.5.2 Algorithms and Running Time

All of our encryption schemes, if done properly, will require computers to encrypt and decrypt. (We of course will do examples with absurdly small encryption and decryption spaces, but we will always understand that our words are really hundreds of digits long.) The basic operation of a computer is a flip or a flop or something like that- we will call it a *flop* and by that we will mean the addition or multiplication of two binary digits, or *bit*, and the recording of the outcome.

When a computer computes the sum

$$4 + 23$$

it sees the binary representation

$$100 + 10111$$

and does elementary school addition.

ver. 2015/04/30

This takes it about 5 flops, one for each bit of the two summands, or maybe 2×5 or 3×5 if there is some carrying. I'm not sure of the mechanics. But to add two k bit numbers, it takes $O(k)$ flops.

Multiplying two k bit numbers can be done with $O(k^2)$ flops. (Actually if you are clever about it you can do with about $O(k^{1.45})$ flops. For people doing heavy computation, this can be an important difference. For us it is not so important.)

Problem 2.5. Show that multiplying two k bit numbers can be done in $O(k^2)$ flops.

We define an *algorithm* a little informally as a series of steps to complete a computational task. We will see, for example, the Euclidean Algorithm for finding the GCD of two numbers. An algorithm takes an input I, of one or more numbers, and returns an answer. The *running time* of the algorithm A will be a function $f_A : \mathbb{N} \to \mathbb{N}$, such that for $k \in \mathbb{N}$, $f_A(k)$ is the maximum number of flops that the algorith A requires to return the anwser for an input I consisting of k or fewer bits.

Where the algorithm A is the elementary-school algorithm for addition, (there may be many different algorithms for addition), we have that $f_A(k) = 3k$. Certainly the algorithm may take fewer steps. If one of the input digits is a string of k zeroes it only takes k flops. But f_A measures the worst case.

We have said that the standard addition algorithm has running time $O(k)$ and the standard multiplication algorithm has running time $O(k^2)$. It is not hard to see that standard algorithms for subtraction and division have have running time $O(k)$ and $O(k^2)$ respectively. We say that the running times are *linear* and *quadratic* respectively.

These are our basic operations. We will show later that the Euclidean algorithm has running time $O(k^3)$. We will do this by observing that it is 'linear in division', so has running time $O(k^3)$.

2.5.3 How about base 10?

Computers use base 2, and we think of our algorithms in these terms, but we will find it useful to talk in base 10. We may compute the running time of an algorithm as a function of the input integer n, instead of the number of binary (base 2) digits.

If the input is n, then it has $k = \log_2 n$ binary digits. When we talk of the running time of the algorithm, an n will always stand for the input integer, and k will always stand for the number of binary digits in the input integer.

So our addition algorithm has running time $O(k)$ or $O(\log_2 n)$, and our multiplication algorithm has running time $O((\log_2 n)^2)$.

ver. 2015/04/30

2.5.4 Problem Complexity

A *computational problem* or just *problem* is basically a function that can be computed algoritmically. Some examples of problems are addition (ADD) and multiplication (MULT). A problem has input and outpt. For ADD the input is two numbers a and b; the output is $a + b$. The set of input, a and b, is an *instance* of the problem. Another problem we will see is primality (PRIME). And instance of PRIME is an integer n. The output is 'true' if n is prime, and 'false' if n is composite. Problems whose output is true or false are called *decision problems*.

An algorithm A_P *solves a problem* P if for any instance of the problem it produces and output in finite time.

The *complexity* $\text{Comp}(A)$ of a problem A is the minimum of the running times of all the algorithms that solve A. As running time is, complexity is a function of k, the number of bits of the instance, but will occasionally be given as a function of n.

We have seen that $\text{Comp}(ADD) = O(k)$ and $\text{Comp}(MINUS) = O(k)$. Also $\text{Comp}(MULT) = O(k^2)$ and $\text{Comp}(DIV) = O(k^2)$.

Problem 2.6. If $\text{Comp}(A) = O(k^2)$ and $\text{Comp}(B) = O(k^2)$ for problems A and B, we might be tempted to say $\text{Comp}(A) = \text{Comp}(B)$. Why is this not okay?

2.5.5 Easy vs. Hard

Addition is linear and multiplication is quadratic. We usually don't even care about this difference. They are both polynomial, and compared to an exponential function this difference is insignificant.

When we have an encryption scheme, there are a couple of algorithms involved. The main once are encryption, decryption, and cryptoanalasis (decryption without the key). Easy and hard will be relative terms. Usually, encrytption and decryption will be done with polynomial algoritms. We call the encryption and decryption problems *easy*. The cryptoanalysis problem will require an exponential algorithm. We call this *hard*.

Occasionally though, we will have polynomial encryption and cryptoanalysis that is not quite exponential. It will still be in order more than polynomial though and this will be enough. In this context we will call this hard.

There is one difficulty in calling a problem hard. To do so, we usually want to show that it cannot be solved with an algorithm having polynomial complexity. Most of the time, this turns out to be very difficult to do.

Perhaps you have heard of the famous open problem of 'P vs. NP'. The essence of it is that there are great number of decision problems for which

ver. 2015/04/30

there is no known polynomial time algorithm, but we cannot prove that they don't have a polynomial time algorithm. For many of them, in particular, those problems known as NP-complete (NPC) problems, the best known algorithm is exponential. We cannot prove that these problems will not be solved in polynomial time, but most mathematicians believe this. When we say a problem is hard, we mean that the best known algorithm for it is hard, usually exponential, and in a stronger sense that the problem is at least as hard as some problem that is well known to be hard.

Note

Actually: Decision problems are in the class NP (Non-deterministic Polynomial) if there is a polynomial time 'certificate' for true instances. One such problem is three colouring a graph. Given a graph, if there is a three colouring, the certificate is the three colouring. We can check in poynomial time if a colouring of the vertices is a good three colouring. The reason the problem is not polynomial (or we think it isn't) is that there are exponentially many possible colours that we have to check. Sometimes, only one of them is a good three colouring. We have a 'polynomial certificate' for true instance but no polynomial algorithm for the problem.

A problem is NP-complete if it is 'the hardest' problem in NP. We look at what this means now.

2.5.6 Polynomial Reductions

Say we have a problem A whose complexity we would like to determine. If the complexity is not obviously polynomial, this is difficult to do. If we think that A is a 'hard' problem, we generally won't be able to prove it is not polynomial. What we do though, is show that it is not much easier than some other problem B which we know is 'hard'.

Actually, we show that B is not much harder than A.

We do this as follows. We pretend we have an algorithm for A, with complexity $f_A = \mathrm{Comp}(A)$.

We then find an algorithm for B that calls A as a function. We can calculate the running time of this algorithm in terms of k and the complexity of A.

Say, for example, that we find an algorithm that for a k bit instance of B requires k^2 basic operations and k calls to function for A, each time with a $2k$ bit input.

Then the algorithm has running time $f(k) = k^2 + k \cdot f_A(2k)$.

Problem 2.7. Show that if f_A is polynomial then $f(k)$ is polynomial. Show that if $f(k)$ is exponential, then f_A is exponenial.

We don't know $\mathrm{Comp}(A)$, but we pretend it is linear: that $f_A(k) = k$. The algorithm is called an *linear oracle* for A. Assuming we have one we may replace

ver. 2015/04/30

$f_A(2k)$ in $f(k)$ above with $2k$. So using a linear oracle for A, our algorithm for B has running time $f(k) = k^2 + k \cdot 2k = 3k^2$.

A problem B is *polynomially reducible* to a problem A, written $B \leq_{poly} A$, if there is an algorithm for B, using an linear oracle for A, which has polynomial running time.

A and B are *polynomially equivalent*, written $A =_{poly} B$ if $A \leq_{poly} B$ and $B \leq_{poly} A$.

A decision problem is NP-complete if and only if it is polynomially equivalent to another NP-complete problem, if and only if it is polynomially equivalent to graph 3-colouring.

1.7.7 Back to encryption schemes

(This is after an interlude into complexity. See the complexity notes before this.)

In our public key encryption schemes, we have an encryption key e_k and a decryption key d_k. As a rule of thumb, the scheme is good if e_k and d_k are polynomial, but cryptoanalysis is exponential.

If encryption has complexity $f(k) = 10k^3 + 1000k$ while cryptoanalysis has complexity $g(k) = .2 \cdot 2^k$, then this isn't so good for blocksize $k = 10$, but choosing k properly, say $k = 60$ we get that $g(60) = 2 \times 10^{17}$ is about 10^{10} times as big as $f(60) \approx 2 \times 10^7$. Today's computers can do 10^7 flops in a second, it would take 100 years to do 10^{17}. So for the right blocksize, this is a good code.

Similarily, there can be good codes, according to our definition, that are good, but the blocksize necessary to ensure that cryptoanalysis is impossible also makes encryption impossible for today's computers. We still call them good though, because when computers get fast enough, they will become useful.

Problem 1.8. Explain this in a bit more detail.

1.2 Divisibility and GCDs

Recall that for integers a and b, we we say that a *divides* b, and write $a \mid b$, if there is another integer q, called the *quotient*, such that $aq = b$. If a divides b, then a is a *divisor* or *factor* of b, and b is a *multiple* of a.

Where all letters involved are integers it should be simple from this definition to prove such properties

i). $a|0$

ii). $0|a \Rightarrow a = 0$

ver. 2015/04/30

iii). $1|a$

iv). $a|1 \Rightarrow a = \pm 1$

v). $a|b_1$ and $a|b_2 \Rightarrow a|c_1b_1 + c_2b_2$

vi). If $b + c = d$, and a divides two of b, c, d then it divides all of them.

With a bit more work you should be able to prove the the Division Algorithm: that for two integers a and b there is a unique pair of integers q and r with $0 \leq r < a$ such that

$$a = b \cdot q + r.$$

In this case r is called the *remainder* upon division of a by b.

The greatest common divisor $\gcd(a, b)$ of two integers a and b is the greatest integer that divides them both. From the last of the six properties given above, and the Division Algorithm, you should be able to find the gcd of two integers using the *Euclidean Algorithm.*

Beginning with integers $a_1 \geq b_1$. At step i we set $a_i = b_{i-1}$, $b_i = r_{i-1}$ and divide a_i by b_i to get a remainder r_i. The algorithm stops when $r_i = 0$, and returns b_i as the GCD of a_0 and b_0.

Example 1.2. To find $\gcd(1253, 234)$ we use the Euclidean Algorithm:

Step	a	$=$	b	$\cdot$	q	$+$	r
1	1253	=	234	·	5	+	83
2	234	=	83	·	2	+	68
3	83	=	68	·	1	+	15
4	68	=	15	·	4	+	8
5	15	=	8	·	1	+	7
6	8	=	7	·	1	+	1
7	7	=	1	·	7	+	0

So $\gcd(1253, 234) = 1$. Now to analyse the time this algorithm takes, there are two things that we consider. One is how many steps the algorithm requires, and the other is how long each step is.

In a typical step, we divide an integer a_i by a smaller integer b_i then multiply by q_i and subtract from a_i to get r_i. Perhaps one can do this more efficiently, but with this, the ith step has complexity $O((\log_2 a_i)^2)$.

Where k_i is the number of binary digits of a_i, without leading zeroes, step i has complexity $O(k_i^2)$.

Now the number of steps depends on $b_1 = a_2$, we show the following.

Claim 1.3. *The algorithm takes at most* $2k_2 = 2\log_2(b_1)$ *steps.*

ver. 2015/04/30

Proof. Observe that

$$r_i = r_{i+1} \cdot q_i + r_{i+2}$$

where we use b_1 for r_0. So $r_{i+2} < r_i/2$. That is, r_i is reduced by at least one binary digit every two steps. Thus $r_t < 1$ by the time $t = 2\log_2(r_0) = 2\log_2(b_1) = 2k_2$. ◇

So the GCD algorith takes $O(k_2)$ steps, having complexities $O(k_1^2)$ then $0(k_2^2)$, then $O(k_2^2)$, then $O(k_3^2)$, then $O(k_3^2)$, etc. We know that $k_i \leq k_{i-2} - 1$, so where $k = k_2$ this is bounded in order by

$$k_1^2 + 2(k^2 + (k-1)^2 + \cdots + 1^2).$$

But we know that $\sum_{i=1}^{k} i^2 \approx k^3$, so the algorithm has complexity $O(k_1^2 + k_2^3)$. Usually we assume that a and b are about the same size though, and say that GCD is cubic, having complexity $O(k^3)$.

As $\gcd(a, b)$ divides a and b, then it divides any linear combination $c_1a + c_2b$ of them for integers c_1 and c_2. Using this *Extended Euclidean Algorithm*, you should be able to write $\gcd(a, b)$ as a linear combination $\gcd(a, b) = c_1a + c_2b$ of a and b. So alternately we could define $\gcd(a, b)$ as the smallest positive linear combination of a and b with integer co-efficients.

Problem 1.9. Using the Extended Euclidean Algorithm (read Section 1.2 if you haven't seen it) express 1 as a linear combination of 1253 and 234.

Problem 1.10. Show that the Extended Euclidean Algorithm has complexity $O(k^3)$, (where we assume a and b both have about k binary digits).

1.3 Modular Arithmetic

You've probably seen Modular Arithmetic before, so we skip many details. The text fills in the details if you are unfamiliar.

Fixing an integer m, we say integers a and b are *congruent mod m*, and write $a \equiv b \mod m$ if $m \mid (a - b)$. Alternately, a and b are congruent mod m if and only if there exists some integer c such that $a = cm + b$.

Example 1.4.

$$-2 \equiv 5 \equiv 12 \equiv 712 \mod 7$$

Some basic properties of Modular Arithmetic are the following.

If $a_1 \equiv b_1$ and $a_2 \equiv b_2 \mod m$ then

i). $a_1a_2 \equiv b_1b_2 \mod m$,

ii). $a_1 + a_2 \equiv b_1 + b_2 \mod m$,

ver. 2015/04/30

iii). $a_1 - a_2 \equiv b_1 - b_2 \mod m$.

Problem 1.11. Prove these properties.

One sees that $a \equiv r \mod m$ where r is the remainder upon division of a by m:

$$a = q \cdot m + r$$

So every integer is congruent modulo m to some integer in $\{0, 1, \ldots, m-1\}$. In fact, reducing a to this remainder r is the canonical quotient map that takes the ring $\mathbb{Z}$ to the quotient ring $\mathbb{Z}/m\mathbb{Z} = \{0, 1, \ldots, m-1\}$ *modulo* the ideal $m\mathbb{Z}$.

The properties you proved in the above exersice are simply showing that the 'remainder' map is a ring homomorphism of $\mathbb{Z}$ to $\mathbb{Z}/m\mathbb{Z}$.

We say that b is a *multiplicative inverse* of a modulo m, if $ab \equiv 1 \mod m$. If a has a multiplicative inverse, we say it is *invertible modulo m.*

For example in $\mathbb{Z}/10\mathbb{Z}$, 7 has an inverse,

$$7 \cdot 3 = 21 \equiv 1 \mod 10,$$

but 5 does not, as all multiples of 5, $0, 5, 10, 15, 20, 25$ are congruent to 0 or 5 modulo 10.

Proposition 1.5. *Let $m \geq 1$ be an integer.*

i). If a is invertible modulo m, its inverse is unique modulo m, that is, if b and b' are inverses of a, then $b \equiv b' \mod m$.

ii). An integer a is invertible modulo m if and only if it is relatively prime to *m; that is, if and only if* $\gcd(a, m) = 1$.

Proof. The first property can be shown directly, or by observing that a has a multiplicative inverse modulo m if and only if the image of a in $\mathbb{Z}/m\mathbb{Z}$ is a unit.

For the second property, see Prop 1.13 of the text. □

In light of the first property, if a is invertible modulo m, we denote its inverse by a^{-1} or $1/a$. To calculate with it, we may replace it with any inverse of a modulo m. We tend to use the one in the range $1, \ldots, m-1$ and call this the inverse.

Example:

$$4/7 = 4 \cdot 7^{-1} \equiv 4 \cdot 3 = 12 \equiv 2 \mod 10.$$

Problem 1.12. Is 2536 invertible modulo 353? If so, what is its inverse?

The set $(\mathbb{Z}/m\mathbb{Z})^*$ of units of $\mathbb{Z}/m\mathbb{Z}$ form a group. The *order* $|(\mathbb{Z}/m\mathbb{Z})^*|$ of this group of units will be important. It occurs enough that there is a function to describe it– *Euler's phi function:*

$$\phi(m) = ||(\mathbb{Z}/m\mathbb{Z})^*|.$$

ver. 2015/04/30

Problem 1.13. What is $\phi(32)$? What is $\phi(6)$? What is $\phi(p \cdot q)$ where $p \neq q$ are prime?

In several applications, we will be computing large powers of an integer a in $\mathbb{Z}/m\mathbb{Z}$. To compute powers of integers a^b is exponential in a and b: it must be, even writing out a^b is exponential. Modulo m however, the size of a^b is essentially the same as the size of a. It is about m. If we compute 7^{415} and then reduce it modulo 3317, then we have to do 415 multiplications of increasingly large numbers. It seems clear that we will save time and space reducing the product at each step.

Problem 1.14. Compute 7^5 modulo 11.

But we still have to to b multiplications of two integers of size m. So the algorithm has $O(b)$ mutliplications of integers of size m. But this is still $O(2^{k_b} k_m^2)$, which is exponential in the number k_b of binary digits of b.

We can do better. Observe that $415 = 2^8 + 2^7 + 2^6 + 2^2 + 2 + 1$. So

$$7^{415} = 7^{2^8+2^7+2^6+2^2+2+1} = 7^{2^8} \cdot 7^{2^7} \cdot 7^{2^6} \cdot 7^{2^2} \cdot 7^2 \cdot 7.$$

It takes us $7 \approx \log_2(415)$ multiplications to evaluate all the factors 7^i modulo 3317, and about the same to multiply them together. This is 7 verses the 415 operations required to so it stupidly. This is called the fast powering algorithm.

Problem 1.15. What is the complexity of the fast powering algorithm?

Problem 1.16. Write a program in Sage to do fast powering. (Sage, of course, uses fast powering, but write it out yourself. The worksheet on the website does a slow powering algorithm.)

1.4 Prime numbers, Unique Factorisation, and finite fields

Recall that an integer $p \geq 2$ is *prime* if and only if for positive integers d

$$d \mid p \Rightarrow d = 1 \text{ or } p.$$

I expect that you've seen and could prove the following results:

- If a prime p divides ab for integers a and b, then it divides a or b.
- There are infinitely many primes.

The first of these is infact only true of 1 and primes, and their negatives. Further, you should have seen the following. You certainly believe it.

ver. 2015/04/30

Theorem 1.6 (Fundamental Theorem of Arithmetic)**.** *Let $a \geq 2$ be an integer. Then a can be written as a product of primes:*

$$a = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots p_d^{e_d},$$

where each p_i is a distinct prime, and each e_i is a positive integer. Furthermore, except for order, this representation is unique.

This representation is called the *prime factorisation* of the integer a. For a particular prime p_i, the exponent e_i is called the *order of p_i in a*, and is denoted $e_i = \mathrm{ord}_{p_i}(a)$. If a prime p does not occur in in the prime factorisation of a, then $\mathrm{ord}_p(a) = 0$.

Example 1.7. The prime factorisation of 204 is

$$2^2 \cdot 3 \cdot 17,$$

so $\mathrm{ord}_2(204) = 2$, $\mathrm{ord}_{17}(204) = 1$, and $\mathrm{ord}_7(204) = 0$.

For prime p, the numbers $1, \ldots p-1$ are all relatively prime to p, and so we have the following useful fact.

Proposition 1.8. *If p is a prime then the group $(\mathbb{Z}/p\mathbb{Z})^*$ of units of $\mathbb{Z}/p\mathbb{Z}$ consists of the set*

$$\{1, 2, \ldots, p-1\}$$

of non-zero elements.

By Proposition 1.8, we have that for prime p, $\mathbb{Z}/p\mathbb{Z}$ is not just a ring, but a field. From field theory we know that the is a unique field of order p. This field is often denoted $\mathbb{F}_p$.

1.5 Powers and Primitive Roots in Finite Fields

We will use properties of $\mathbb{Z}/p\mathbb{Z} \cong \mathbb{F}_p$ that we know from field theory. Several of these properties, we can prove without much theory though, so we do that.

We have by Lagrange's theorem that ever element of a group has order dividing the order of the group: indeed the cosets of the subgroup generated by the element partition the group into equal parts. It follows that $a^{p-1} = 1$ for every element a in the multiplicitive group $(\mathbb{Z}/p\mathbb{Z})^*$, which has order $p-1$. This implies the following theorem, known as Fermat's Little Theorem; but we will prove it without applying Lagrange.

Theorem 1.9. *Let p be prime, and a be an integer. Then $a^{p-1} \equiv 0 \mod p$ if $p \mid a$ and otherwise $a^{p-1} \equiv 1 \mod p$.*

ver. 2015/04/30

Proof. If $p|a$ then $a^{p-1} \equiv 0^{p-1} = 0$, so we may assume that $p \nmid a$. As the statement is about the image of a modulo p we may assume that a is its image in $(\mathbb{Z}/p\mathbb{Z})$. By the assumption $p \nmid a$, we then have that a is in $(\mathbb{Z}/p\mathbb{Z})^*$. The elements

$$a, 2a, \ldots (p-1)a$$

are distinct elements, for if $ia \equiv ja$ then $p \mid j - i$ which implies $i = j$. So we have

$$a^{p-1}(1 \cdot 2 \cdot \cdots \cdot p-1) = a \cdot 2a \cdot \cdots \cdot (p-1)a = 1 \cdot 2 \cdot \cdots \cdot (p-1).$$

Thus $a^{p-1} = 1$ in $(\mathbb{Z}/p\mathbb{Z})^*$, which means that $a^{p-1} \equiv 1 \mod p$. □

Immediate corollaries of this are the fact that for a with $\gcd(a, p) = 1$, we have

- $a^p \equiv a \mod p$
- $a^{-1} \equiv a^{p-2} \mod p$

This second seems nicer than the Extended Euclidean Algorithm for computing inverses modulo p. Conceptually it is easier, but computationally it is about the same: $2\log_2 p = O(k)$ multiplication/modular reductions of two k bit numbers, so $O(k^3)$.

The theorem also gives a nice check that a number is prime. Given an integer n the theorem gives us that if n is prime then for any smaller a, $a^{n-1} \equiv 1 \mod n$. So if we compute $2^{n-1} \mod n$ and get anything other than 1, then n is not prime. And we didn't have to find factors. It could be that we get a 1 when n is not prime, but doing the same calculation with a couple of different bases in place of 2, we can by pretty confident about whether or not n is prime. (However, this test is not perfect: there are known composite numbers n for which any a that is relatively prime to n yields $a^{n-1} \equiv 1 \mod n$. Such numbers are called Carmichael numbers.) We will look at better prime verification algorithms later.

Definition 1.10. For an integer a, the *order of a modulo p*, is its order in group $(\mathbb{Z}/p\mathbb{Z})^*$ that is, it is the minimum positive integer k such that $a^k \equiv 1 \mod p$.

Fermat's Little Theorem also gives us the following, (which you likely also know from group theory.)

Proposition 1.11. *Let p be a prime, and a be an integer not divisible by p. If $a^n \equiv 1 \mod p$ then the order of a modulo p divides n. In particular, it divides $p - 1$.*

Proof. See text. This is Prop 1.30. □

ver. 2015/04/30

Now we know from field theory that any finite field has a cyclic multiplicative group. This is too much to prove here, even for finite fields $\mathbb{F}_p$ of prime order, (there are also finite fields of prime-power order), but we will use it.

Theorem 1.12 (The Primitive Root Theorem)**.** *Let p be a prime number. Then there exists an element $g \in \mathbb{F}_p^*$ such that*

$$\mathbb{F}_p^* = \{1, g, g^2, \ldots, g^{p-2}\}.$$

The elements g of the theorem are called *generators* or *primitive roots* of $\mathbb{F}_p^*$.

Problem 1.17. Find a primitive root of $\mathbb{F}_7^*$. How many are there?

You might do this by taking a element a to every power in $1, 2, \ldots, 6$, but you don't really have to do this. You only have to check certain powers. Which ones? In Excercise 1.30 from the text, you will have to do this for a prime p much bigger than 7, so think about this.

One can show that for k dividing $p-1$ there are exactly $\phi(k)$ elements in $\mathbb{F}_7^*$ having order k. You will do so in homework Question 1.31 of the text, only we don't know we are doing this yet. We will address properties of ϕ, later.

Notice that all the results of this section can easily be proved if we assume the Primitive Root Theorem.

Though $\mathbb{Z}/p\mathbb{Z}$ and $\mathbb{F}_p$ are isomorphic and the structure is clear when representing the field as $\mathbb{F}_p = \{0, 1, g, g^2, \ldots, g^{p-2}\}$ for some primitive root g, what we will often use is the fact that the structure in terms of $\mathbb{Z}/p\mathbb{Z}$, is not clear.

More concretely, we can fix an isomorphism $f : \mathbb{F}_p^* \to (\mathbb{Z}/p\mathbb{Z})^*$ by choosing a primitive root $f(g)$ in $(\mathbb{Z}/p\mathbb{Z})$. The sequence $f(g), f(g^2), f(g^3), \ldots, f(g^{p-1})$ list the elements of $(\mathbb{Z}/p\mathbb{Z})^*$ but mixes them up pretty good. (See Figure 2.2 from page 65 of the text.) One of the properties we will take advantage of is that there isn't any fast way to predict what integer x gives $f(g^x) = h$ for a number h in $(\mathbb{Z}/p\mathbb{Z})^*$.

Because of this we will use both $\mathbb{F}_p$ and $\mathbb{Z}/p\mathbb{Z}$. We will treat them notationally differently, but will exploit the fact that the are isomorphic.

ver. 2015/04/30

2 Discrete Logarithms and Diffe Hellman

Section 2.1 is history. Read it if you are interested.

2.1 The Discrete Logarithm Problem

Definition 2.1. Let g be a primitive root of $\mathbb{F}_p$. For any element $h \in \mathbb{F}_p^*$, the *discrete log of h (in the base g)* is the solution $\log_g(h)$ modulo p to the equation

$$g^x = h.$$

Problem 2.1. Why is there necessarily a solution, and why is it unique modulo $p-1$?

The discrete log $\log_g : \mathbb{F}_p^* \to (\mathbb{Z}/p\mathbb{Z})^*$ is a one-to-one function. As we mentioned before, viewed as as function $\log_g : (\mathbb{Z}/p\mathbb{Z})^* \to (\mathbb{Z}/p\mathbb{Z})^*$, $\log_g$ mixes things up pretty good.

Indeed, take a look at the Chapter 2 Sage worksheet for an example.

Now how do we evaluate the discrete log function.

Problem 2.2. Find $\log_g(g^3)$ in $\mathbb{F}_{13}^*$

Ah! Its 3, maybe that was too easy.

Example 2.2. 2 is a primitive root of $\mathbb{F}_{13}^*$. Lets find $\log_2 9$.

The brute force way to do this is start computing $2, 2^2, 2^3 \ldots$:

$$2, 4, 8, 3, 6, 12, 11, 9, 5, 10, 7.$$

We see that 9 is the 8th power, so $\log_2(9) = 2$.

We can do this in $\mathbb{F}_{13}^*$, but its bad algorithm. We could have stopped after writing down 9, but computing $\log_g(a)$ in $\mathbb{F}^*(p)$ like this takes $O(p) = O(2^{k_p})$ multiplications (each $O(k_p^2)$). This is much longer than the $O(k_p)$ multiplications of fast powering.

We can do better than $O(2^k)$, but are not able to do anywhere near $O(k)$.

By the way, we mentioned above that 2 was a primitive root of $\mathbb{F}_{13}^*$. How would we show this?

Problem 2.3. Show that 10 is a primitive root of $\mathbb{F}_{47}$. Compute $\log_{10}(22)$.

> **Note**
>
> Showing that 10 generates $\mathbb{Z}/47\mathbb{Z}$ **additively** is simple, as it is relatively prime to p. Showing that it is a primitive root is showing that it generates $(\mathbb{Z}/47\mathbb{Z})^*$, (multiplicatively!).

ver. 2015/04/30

Okay, you did that. Did you write out all 46 powers of 10 in $\mathbb{F}_{47}$. You might have done better to start by factoring $46 = 2 \cdot 23$. If 10 is not primitive, it has order dividing 46. So we only have to show that $10^2 (= 100 = 6)$ and $10^{23} = (10^{16} * 10^4 * 10^2 * 10)$ are not 1.

This seems a better way. But it is not so good either: factoring is hard! So finding a primitive root seems tricky right now. It is something we will want to do, but we don't need it to defined the discrete log.

Definition 2.3. Let G be a (multiplicative) group and g be an element of G. The *discrete log problem (DLP(G,g))* for G is

$$\textbf{Input} : anelementhofG$$

ver. 2015/04/30